Connect the Existing Website to MySQL with FastAPI

Team 59 - Smart AI Planner
Step-by-step team guide | October 8, 2026

Keep the existing website and database. Build a small Python backend between them. The website sends requests; FastAPI runs SQL queries and returns data.

```
Existing website <-> FastAPI <-> SQLAlchemy + PyMySQL <-> MySQL
```

Scope: This guide provides a local connectivity prototype for reading and adding tasks. It does not include authentication. Use test data and keep the prototype bound to your own computer. Connect the existing login system before deployment. The example has not been tested against your team's database.

1. Collect the existing database information

Ask the database teammate for: MySQL host, port (usually 3306), database name, a backend database account, the task table name, column definitions, and any remote TLS requirements. Share passwords privately.

In MySQL Workbench, select the existing database and run these commands. Replace **your_task_table** with the real table name:

```
SHOW TABLES;
DESCRIBE your_task_table;
SHOW CREATE TABLE your_task_table;
```

Do not create new tables or replace the database. Use a development copy or agreed test records. Give the backend account only the required permissions. If MySQL is on another computer, use its reachable address; 127.0.0.1 means the computer running FastAPI.

2. Identify the website connection points

The website teammate identifies the code displaying tasks, the Add Task form, the local website address (for example http://localhost:5173), and the existing login system. Replace sample data with API responses. Use the website's existing screens and controls.

3. Create the backend folder

Install Python 3.11 or newer. In a Windows terminal, run these commands individually:

```
mkdir planner-backend
cd planner-backend
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install fastapi uvicorn sqlalchemy "pymysql[rsa]" python-dotenv
```

Create three files in planner-backend: **.env**, **.gitignore**, and **main.py**. FastAPI supplies the API, SQLAlchemy runs database queries, and PyMySQL connects to MySQL.

4. Add database connection settings

Put this in **.env**. Replace the placeholders and use the actual website origin:

```
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=YOUR_EXISTING_DATABASE_NAME
DB_USER=YOUR_BACKEND_DATABASE_USER
DB_PASSWORD=YOUR_DATABASE_PASSWORD
FRONTEND_ORIGIN=http://localhost:5173
```

Put this in **.gitignore**:

```
.env
.venv/
__pycache__/
```

Keep .env on the backend. Never put MySQL credentials in browser JavaScript or commit them to the shared repository.

5. Map existing columns to API fields

The example API uses **id**, **title**, and **due_date**. The code assumes a table named **tasks** with these columns. Change the SQL names to match the actual database. Keep the API field names consistent for the website.

For example, if the existing table is student_tasks with task_id, task_name, and deadline:

```
SELECT task_id AS id, task_name AS title, deadline AS due_date
FROM student_tasks
ORDER BY deadline, task_id
LIMIT 100;

INSERT INTO student_tasks (task_name, deadline)
VALUES (:title, :due_date);
```

Include other required columns in the insert. A required user_id must ultimately come from verified login. The example assumes an auto-increment integer ID and a required date-only deadline. Adapt types, nullability, and title length to the actual schema.

6. Add the FastAPI code - main.py (part 1 of 2)

Paste both parts into the same **main.py** file, in order. This local prototype has no login. Use only test data and do not expose it publicly.

```
import os
from datetime import date

from dotenv import load_dotenv
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, Field
from sqlalchemy import URL, create_engine, text

load_dotenv()

database_url = URL.create(
    "mysql+pymysql",
    username=os.environ["DB_USER"],
    password=os.environ["DB_PASSWORD"],
    host=os.environ["DB_HOST"],
    port=int(os.environ["DB_PORT"]),
    database=os.environ["DB_NAME"],
    query={"charset": "utf8mb4"},
)

engine = create_engine(
    database_url,
    pool_pre_ping=True,
    pool_recycle=1800,
)

app = FastAPI(title="Team 59 Planner API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=[os.environ["FRONTEND_ORIGIN"]],
    allow_methods=["GET", "POST"],
    allow_headers=["Content-Type"],
)


class TaskCreate(BaseModel):
    title: str = Field(min_length=1, max_length=200)
    due_date: date


class TaskOut(BaseModel):
    id: int
    title: str
    due_date: date


@app.get("/health")
def health():
    with engine.connect() as connection:
        connection.execute(text("SELECT 1"))
    return {"database": "connected"}
```

6. Add the FastAPI code - main.py (part 2 of 2)

```
@app.get("/api/tasks", response_model=list[TaskOut])
def read_tasks():
    # Match these table and column names to the existing schema.
    with engine.connect() as connection:
        rows = connection.execute(
            text("""
                SELECT id, title, due_date
                FROM tasks
                ORDER BY due_date, id
                LIMIT 100
            """)
        ).mappings().all()
        return [dict(row) for row in rows]


@app.post("/api/tasks", response_model=TaskOut, status_code=201)
def create_task(task: TaskCreate):
    title = task.title.strip()
    if not title:
        raise HTTPException(
            status_code=422,
            detail="Task title cannot be blank.",
        )

    # Match names and add any other required database columns.
    with engine.begin() as connection:
        result = connection.execute(
            text("""
                INSERT INTO tasks (title, due_date)
                VALUES (:title, :due_date)
            """),
            {"title": title, "due_date": task.due_date},
        )
        return {
            "id": result.lastrowid,
            "title": title,
            "due_date": task.due_date,
        }
```

The values :title and :due_date are parameters. Keep user input separate from SQL commands. engine.begin() commits a successful insert and rolls it back if the operation fails.

7. Start FastAPI and verify MySQL access

From the backend folder, run the following and keep the terminal open:

```
.\.venv\Scripts\python.exe -m uvicorn main:app --reload --host 127.0.0.1 --port 8000
```

Open **http://127.0.0.1:8000/health**. Expected response:

```
{"database": "connected"}
```

This confirms the backend reached MySQL. If it fails, fix the database settings before proceeding.

8. Test reading and writing before the website

Open **http://127.0.0.1:8000/docs**. This is FastAPI's interactive API testing page.

Read: Expand GET /api/tasks, click Try it out, then Execute. Confirm the expected test records appear. An empty list [] means the query succeeded but returned no rows.

Write: Expand POST /api/tasks, click Try it out, and enter:

```
{
  "title": "API connection test",
  "due_date": "2026-10-26"
}
```

Click Execute. Expect HTTP status 201 and a saved task ID. Run GET /api/tasks again. Have the database teammate confirm the new row in MySQL Workbench. If a required field has no default, add it to the request model and insert query.

9. Connect the existing task list

Put these functions in the website's existing JavaScript or API service file:

```
const API_BASE = "http://127.0.0.1:8000";

async function getTasks() {
  const response = await fetch(`${API_BASE}/api/tasks`);
  if (!response.ok) {
    throw new Error("Could not load tasks.");
  }
  return response.json();
}

async function loadTasks() {
  try {
    const tasks = await getTasks();
    // Replace with the existing task-list display function.
    console.log(tasks);
  } catch (error) {
    // Show this in the existing website error area.
    console.error(error.message);
  }
}
```

Call loadTasks() where the website currently loads sample tasks. Replace console.log(tasks) with the actual rendering function. For React, if the existing state setter is setTasks:

```
const tasks = await getTasks();
setTasks(tasks);
```

Keep the website's existing design. Only change where its data comes from.

10. Connect the existing Add Task form

Add this function to the same website API service or JavaScript file:

```
async function createTask(title, dueDate) {
  const response = await fetch(`${API_BASE}/api/tasks`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title, due_date: dueDate })
  });
  const data = await response.json();
  if (!response.ok) {
    throw new Error(
      typeof data.detail === "string"
        ? data.detail
        : "Could not save the task. Check the form."
    );
  }
  return data;
}

async function handleTaskSubmit(event) {
  event.preventDefault();
  try {
    // Replace these selectors with the actual input selectors.
    const title = document.querySelector("#task-title").value;
    const dueDate = document.querySelector("#task-due-date").value;
    await createTask(title, dueDate);
    await loadTasks();
  } catch (error) {
    // Show this in the website's existing error area.
    console.error(error.message);
  }
}
```

For plain HTML, attach the handler to the actual existing form:

```
document
  .querySelector("#task-form")
  .addEventListener("submit", handleTaskSubmit);
```

For React or another framework, use its existing form handler and state values instead. Do not add a second form. The selectors above are placeholders for the existing website.

Run the website through its development server, rather than double-clicking an HTML file. If CORS fails, set FRONTEND_ORIGIN to the exact website address, including the port, and restart FastAPI. localhost and 127.0.0.1 are different origins.

11. Connect the existing login before real student data

The prototype proves connectivity but does not restrict task access. For the final API:

1. Identify the website's actual login system.
2. Verify its token or session in FastAPI on every task request.
3. Map the verified identity to the existing database user.
4. Filter task reads by that user.
5. Include that verified user ID when inserting tasks.
6. Check both task ID and user ID for updates and deletes.

```
SELECT id, title, due_date
FROM tasks
WHERE user_id = :verified_user_id;
```

Adapt names to your schema. Never trust a user_id supplied by a form. Authentication code depends on the existing login system. A visual login screen alone is not authentication. CORS does not check identity. If bearer tokens are used, also allow Authorization in CORS headers and send the token with website requests.

12. Deploy the connection

Deploy FastAPI to a Python-capable server. Set database credentials privately there. Permit MySQL access from that backend server, use verified TLS for remote MySQL, and give the API an HTTPS address. Update the website's API_BASE and the backend's allowed origin. Run without development reload mode. Keep MySQL inaccessible to ordinary website visitors.

```
const API_BASE = "https://api.your-team-domain.com";
```

On each teammate's computer, 127.0.0.1 points to that computer. Team and deployed testing require a reachable backend address. Add appropriate authentication protections, backups, and error monitoring before public use.

13. Confirm the finished connection

Demonstrate all six checks:
1. Existing tasks load from MySQL into the website.
2. A website submission creates the correct MySQL record.
3. Refreshing the website loads that saved record.
4. Invalid data produces a helpful error.
5. Requests without valid login are rejected.
6. One user cannot read or change another user's tasks.

Work order: Database teammate supplies schema and access; backend developer implements the API; website teammate connects existing controls; security and hosting teammates verify identity, network access, and deployment.

Troubleshooting

Access denied
Check the MySQL username, password, and permitted connecting host.

Connection refused
Check the MySQL service, host, port, firewall, and network access.

Unknown database
Check DB_NAME in .env.

Missing table or column
Compare query names with SHOW CREATE TABLE output.

Field has no default value
Supply every required insert field. User ownership must come from verified login.

CORS error
Match the exact website origin and restart the backend.

API works but website is empty
Check browser network responses, field names, and the existing display function.

Response validation error
Check column types, missing aliases, and nullable fields against TaskOut.

Works only on one computer
Replace local addresses with reachable server addresses and check network access.

Official reference documentation

SQLAlchemy: MySQL and PyMySQL
https://docs.sqlalchemy.org/en/20/dialects/mysql.html

FastAPI: CORS configuration
https://fastapi.tiangolo.com/tutorial/cors/

FastAPI: authentication example
https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/

Completion target: Existing website controls call FastAPI; FastAPI accesses the existing MySQL schema; authenticated users access only their own records.